

Week 2 · Day 14 (Capstone): Orchestration & MLOps

Airflow, KubeFlow, LoRA Deployment, Continuous Fine-Tuning & End-to-End Architecture

Capstone Goal: Wire everything into one automated pipeline -- data in, model out, monitored in production. By end of day you will have a working Airflow DAG that runs nightly evals and promotes models on quality gate pass, a KubeFlow pipeline that takes labelled data to a served LoRA adapter, and a hot-swap mechanism that deploys the new adapter without restarting vLLM.

Week 2 wrap-up instruction: After completing this day, draw your own end-to-end architecture diagram from memory. Include every component covered in the two-week curriculum: inference runtime, KV cache, quantization, containerisation, observability, caching layers, API gateway, distributed parallelism, and the MLOps pipeline. If you can draw it from memory, you own the material.

End-to-End Pipeline Architecture

The complete production LLM system has three interconnected planes:

Plane	Components	Orchestrated by
Serving plane	vLLM + PagedAttention + Flash Attention, API Gateway, Semantic/Prefix cache, Multi-model router, Prometheus+Loki+Jaeger	Kubernetes + Traefik
Training plane	Data ingest, LoRA fine-tune, Eval harness, Model registry, Artifact store (PVC/S3)	KubeFlow Pipelines
Ops plane	Nightly eval regression, Quality gate, Auto-promote, Slack alerting, A/B traffic split, Cost dashboard	Airflow DAGs

PART A — Apache Airflow: Nightly Eval & Promote DAG

Why Airflow for LLM MLOps?

Airflow schedules and orchestrates workflows as Directed Acyclic Graphs (DAGs). For LLM systems the critical nightly workflow is: pull new labelled data, run evals against the current production model and any candidate models, compare quality scores, and either auto-promote the candidate or fire a Slack alert if quality has regressed. Airflow handles retries, backfill, dependency tracking, and audit history -- all of which matter when your nightly job gates a production deployment.

A.1 Install and Configure Airflow

```
# Install Airflow with CeleryExecutor for distributed task execution
pip install apache-airflow[celery,redis,postgres]==2.8.0
```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

        num_epochs=num_epochs,
    )
    # Request 1 GPU for this step
    kubernetes.use_field_path_as_env(
        train_step, env_name="MY_NODE_NAME", field_path="spec.nodeName")
    train_step.set_accelerator_type("nvidia.com/gpu").set_accelerator_limit(1)
    train_step.set_memory_limit("120G")

    # Step 3: evaluate adapter (1x A10G GPU)
    eval_step = evaluate_adapter(
        base_model=base_model,
        adapter_model=train_step.outputs["output_adapter"],
        quality_threshold=quality_threshold,
    )
    eval_step.set_accelerator_type("nvidia.com/gpu").set_accelerator_limit(1)

    # Step 4: deploy only if eval gate passed (conditional execution)
    with dsl.If(eval_step.output == True, name="gate-passed"):
        deploy_step = deploy_adapter(
            adapter_model=train_step.outputs["output_adapter"],
            vllm_endpoint=vllm_endpoint,
            adapter_name=adapter_name,
        )

# Compile and upload pipeline
if __name__ == "__main__":
    from kfp import compiler
    compiler.Compiler().compile(lora_pipeline, "lora_pipeline.yaml")
    print("Pipeline compiled to lora_pipeline.yaml")
    print("Upload to KFP UI: http://localhost:8080")

```

PART C — LoRA Adapter Deployment & Hot-Swap

What is LoRA?

LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning technique. Instead of updating all N billion model weights (expensive, slow, large), LoRA freezes the original weights and adds small trainable rank-decomposition matrices to targeted weight matrices (typically Q, K, V, O projections). A rank-16 LoRA adapter for a 8B model adds only ~100MB of trainable parameters versus 16GB for the full model -- a 160x reduction in training cost and adapter size. At inference, the adapter weights can be **merged** into the base model (zero runtime overhead) or kept **separate** for hot-swappable multi-tenant serving.

Approach	Adapter merged?	Runtime overhead	Hot-swap?	Best for
Merge + serve	Yes (permanent)	Zero	No	Single use case, max speed
PEFT adapter	No (dynamic)	Minimal (<5%)	Yes	Multi-tenant, A/B testing
vLLM LoRA API	No (dynamic)	Minimal	Yes	Production multi-adapter serving

C.1 Merge LoRA Adapter with PEFT

```
# Merge adapter into base model weights (static deployment -- fastest inference)
from peft import PeftModel, AutoPeftModelForCausalLM
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch, time

BASE_MODEL      = "meta-llama/Meta-Llama-3-8B-Instruct"
ADAPTER_PATH    = "./lora-adapter-v1"
MERGED_OUTPUT   = "./llama3-8b-finetuned-merged"

print("Loading base model...")
model          = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL, torch_dtype=torch.float16, device_map="cpu")
tokenizer      = AutoTokenizer.from_pretrained(BASE_MODEL)

print("Loading LoRA adapter...")
model = PeftModel.from_pretrained(model, ADAPTER_PATH)

print("Merging adapter into base weights...")
t0      = time.perf_counter()
model = model.merge_and_unload()      # permanently bakes LoRA into weights
elapsed = time.perf_counter() - t0
print(f"Merge completed in {elapsed:.1f}s")

print("Saving merged model...")
model.save_pretrained(MERGED_OUTPUT)
tokenizer.save_pretrained(MERGED_OUTPUT)

# Verify merged model generates correctly
from transformers import pipeline
pipe = pipeline("text-generation", model=MERGED_OUTPUT,
    device_map="auto", max_new_tokens=50)
out  = pipe("[INST] what is KV caching? [/INST]")
```

```
print(out[0]["generated_text"][-200:])
```

C.2 vLLM Dynamic LoRA Adapter API (Hot-Swap)

```
# vLLM 0.4+ supports loading and serving multiple LoRA adapters
# simultaneously without restarting the server
```

```
# Start vLLM with LoRA support enabled
```

```
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
  --enable-lora \                # enable LoRA serving
  --max-loras 4 \                # max adapters loaded simultaneously
  --max-lora-rank 64 \          # max LoRA rank supported
  --lora-dtype float16 \
  --dtype float16 \
  --port 8000
```

```
# Load a LoRA adapter at runtime (no restart!)
import requests
```

```
# Load adapter from local path or HuggingFace Hub
```

```
resp = requests.post("http://localhost:8000/v1/load_lora_adapter", json={
    "lora_name": "customer-support-v1",
    "lora_path": "/models/adapters/customer-support-v1",    # local path
})
print(resp.json())    # {"status": "success", "lora_name": "customer-support-v1"}
```

```
# Load a second adapter (for multi-tenant use case)
```

```
requests.post("http://localhost:8000/v1/load_lora_adapter", json={
    "lora_name": "code-assistant-v2",
    "lora_path": "company/code-assistant-lora",    # HuggingFace Hub path
})
```

```
# List loaded adapters
```

```
resp = requests.get("http://localhost:8000/v1/models")
for model in resp.json()["data"]:
    print(f" {model['id']}")
```

```
# Output:
```

```
# meta-llama/Meta-Llama-3-8B-Instruct (base)
# customer-support-v1 (LoRA adapter)
# code-assistant-v2 (LoRA adapter)
```

[illegible]

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key="vllm")
```

```
# Use base model
```

```
resp_base = client.chat.completions.create(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    messages=[{"role": "user", "content": "What is vLLM?"}],
    max_tokens=80,
)
```

```
# Use customer-support adapter (same server, different weights)
```

```
resp_support = client.chat.completions.create(
    model="customer-support-v1",
    messages=[{"role": "user", "content": "I have an issue with my order."}],
```

```
        max_tokens=80,  
    )  
  
# ■■ Unload adapter when no longer needed  
requests.delete("http://localhost:8000/v1/unload_lora_adapter", json={  
    "lora_name": "customer-support-v1"  
})
```

C.3 Continuous Fine-Tuning Pipeline with Auto-Deploy

```
# continuous_finetune.py -- trigger fine-tuning on new labelled data
# This runs as a KubeFlow pipeline triggered by Airflow when enough new
# labelled examples accumulate (e.g. 500+ new samples)

import requests, json
from datetime import datetime

KUBEFLOW_HOST      = "http://ml-pipeline.kubeflow.svc.cluster.local"
PIPELINE_ID        = "lora-finetune-eval-deploy"
VLLM_ENDPOINT      = "http://vllm-service.inference.svc.cluster.local:8000"
MIN_NEW_SAMPLES    = 500 # trigger threshold

def count_new_labelled_samples(since_ts: str) -> int:
    # Query your data store for samples labelled since last run
    resp = requests.get(f"http://data-api/labels/count?since={since_ts}")
    return resp.json()["count"]

def trigger_finetune_pipeline(adapter_name: str) -> str:
    payload = {
        "pipeline_spec": {
            "pipeline_id": PIPELINE_ID,
        },
        "run_name": f"auto-finetune-{datetime.utcnow().strftime('%Y%m%d-%H%M')}",
        "params": {
            "adapter_name"      : adapter_name,
            "vllm_endpoint"     : VLLM_ENDPOINT,
            "max_samples"       : 2000,
            "quality_threshold" : 0.82, # slightly higher than manual baseline
        }
    }
    resp = requests.post(f"{KUBEFLOW_HOST}/apis/v1beta1/runs", json=payload)
    run_id = resp.json()["run"]["id"]
    print(f"Triggered pipeline run: {run_id}")
    return run_id

def wait_for_run(run_id: str, timeout_minutes: int = 120) -> str:
    import time
    for _ in range(timeout_minutes * 2): # poll every 30s
        resp = requests.get(f"{KUBEFLOW_HOST}/apis/v1beta1/runs/{run_id}")
        status = resp.json()["run"]["status"]
        if status in ("Succeeded", "Failed", "Error"):
            return status
        time.sleep(30)
    return "Timeout"

# Main trigger logic (called by Airflow task)
```

```
def auto_finetune_if_ready(last_run_ts: str, **kwargs):
    n_new = count_new_labelled_samples(last_run_ts)
    print(f"New labelled samples since {last_run_ts}: {n_new}")

    if n_new < MIN_NEW_SAMPLES:
        print(f"Not enough samples ({n_new} < {MIN_NEW_SAMPLES}). Skipping.")
        return "skipped"

    adapter_name = f"lora-v{datetime.utcnow().strftime('%Y%m%d')}"
    run_id = trigger_finetune_pipeline(adapter_name)
    status = wait_for_run(run_id)

    print(f"Pipeline completed with status: {status}")
    return status
```

Capstone: End-to-End Architecture Reference

```
# Complete LLM Production System -- ASCII Architecture Diagram
#
# ████████████████████████████████████████████████████████████████████████████████
# █ CLIENT REQUESTS █
# ████████████████████████████████████████████████████████████████████████████████
#
# █
# ████████████████████████████████████████████████████████████████████████████████
# █ API GATEWAY (Traefik / Kong) █
# █ • Auth (JWT / API key) • Rate limiting (per-key token budget) █
# █ • Request logging -> Loki • SSL termination █
# █ • Load balance across vLLM replicas █
# ████████████████████████████████████████████████████████████████████████████████
#
# █ █
# ████████████████████████████████████████████████████████████████████████████████
# █ ROUTING LAYER █ █ CACHING LAYER █
# █ Multi-model █ █ 1. Redis exact cache (0ms) █
# █ complexity █ █ 2. FAISS semantic cache (5ms) █
# █ classifier █ █ 3. Prompt compression █
# █ (small/large) █ █ (LLMLingua-2) █
# ████████████████████████████████████████████████████████████████████████████████
# █ █
# ████████████████████████████████████████████████████████████████████████████████
# █ vLLM INFERENCE ENGINE (per model tier) █
# █ • PagedAttention (non-contiguous KV blocks, CoW sharing) █
# █ • Continuous batching (step-level scheduling) █
# █ • Flash Attention / FlashInfer (IO-aware, O(Nd) HBM traffic) █
# █ • Prefix KV cache (APC -- shared system prompt blocks) █
# █ • LoRA adapter hot-swap (multi-tenant, zero restart) █
# █ • Speculative decoding (draft model for TTFT reduction) █
# █ ████████████████████████████████████████████████████████████████████████████████ █
# █ █ PARALLELISM (for large models) █ █
# █ █ Tensor Parallel (TP=4, within node, NVLink) █ █
# █ █ Pipeline Parallel (PP=2, across nodes, InfiniBand) █ █
# █ ████████████████████████████████████████████████████████████████████████████████ █
# ████████████████████████████████████████████████████████████████████████████████
#
# ████████████████████████████████████████████████████████████████████████████████
# █ OBSERVABILITY PLANE █
# █ Prometheus (metrics) -> Grafana dashboards █
# █ Loki + Promtail (structured logs) -> LogQL queries █
# █ OpenTelemetry + Jaeger/Tempo (distributed traces) █
# █ Cost tracking middleware ($/hr per model per API key) █
# ████████████████████████████████████████████████████████████████████████████████
#
# ████████████████████████████████████████████████████████████████████████████████
# █ MLOPS PLANE █
# █ Airflow (nightly eval -> quality gate -> promote/alert) █
# █ KubeFlow Pipelines (data -> LoRA fine-tune -> eval -> vLLM deploy) █
# █ Auto-trigger: 500+ new samples -> KFP run -> hot-swap if gate passes █
# ████████████████████████████████████████████████████████████████████████████████
#
# ████████████████████████████████████████████████████████████████████████████████
# █ KUBERNETES CONTROL PLANE █
# █ NVIDIA Device Plugin (GPU allocation) █
```

```
# ■ HPA (scale replicas on vllm_num_requests_waiting metric)      ■
# ■ PVC (model weights, LoRA adapters, eval artifacts)           ■
# ████████████████████████████████████████████████████████████
```

Capstone Verification Checklist

After completing Week 2, verify you can answer and demonstrate each:

```
WEEK_2_CHECKLIST = [
    # Day 8 -- Evals
    ("D08", "Write 20 golden test cases and integrate into CI with pytest",
     "pytest tests/test_model_evals.py --model llama3 -v"),

    # Day 9 -- vLLM
    ("D09", "Serve LLaMA-3 8B with vLLM and measure throughput vs naive HF",
     "python benchmark_vllm_vs_hf.py"),

    # Day 10 -- Attention
    ("D10", "Confirm FlashInfer backend is active in vLLM",
     "vllm serve ... 2>&1 | grep -i flashinfer"),

    # Day 10 -- KV Tuning
    ("D10", "Enable prefix caching and measure TTFT reduction on warm calls",
     "python measure_prefix_cache_benefit.py"),

    # Day 11 -- TP
    ("D11", "Launch vLLM with TP=2 and measure NVLink all-reduce bandwidth",
     "torchrun --nproc_per_node=2 measure_allreduce.py"),

    # Day 11 -- K8s
    ("D11", "Deploy vLLM pod with GPU resource limits and readiness probe",
     "kubectl apply -f vllm-deployment.yaml"),

    # Day 12 -- Caching
    ("D12", "Build semantic cache hitting >30% on paraphrase test queries",
     "python test_semantic_cache.py --queries paraphrase_set.json"),

    # Day 12 -- Gateway
    ("D12", "Verify gateway returns 429 after rate limit and 401 without auth",
     "python test_gateway_auth_ratelimit.py"),

    # Day 13 -- Load test
    ("D13", "Find OOM threshold: highest concurrency before error rate > 5%",
     "python throughput_curve.py --max-users 200"),

    # Day 13 -- Observability
    ("D13", "View a complete trace in Jaeger showing prefill vs decode spans",
     "open http://localhost:16686 -> service=vllm -> operation=decode"),

    # Day 14 -- Airflow
    ("D14", "Run nightly eval DAG manually and verify quality gate decision",
     "airflow dags trigger nightly_eval_promote"),

    # Day 14 -- KFP
    ("D14", "Compile and upload LoRA pipeline YAML to KFP UI",
     "python pipeline.py && kfp run submit --pipeline-file lora_pipeline.yaml"),
]
```



```

# Day 14 -- LoRA
("D14", "Hot-swap a LoRA adapter into running vLLM without restarting",
  "curl -X POST localhost:8000/v1/load_lora_adapter -d '{...}'"),
]

print(f"{'Day':<5} {'Task':<65} {'Verification'}")
print("-" * 130)
for day, task, cmd in WEEK_2_CHECKLIST:
  print(f"{'day':<5} {'task':<65} {'cmd':<55}")

```

Testing & Metrics

MLOps Pipeline Health Metrics

Metric	How to Measure	Target
Eval run success rate	Airflow DAG success rate (last 30 runs)	> 95%
Quality gate pass rate	Promotions / total eval runs	Track trend; alert on sudden drop
Eval-to-deploy latency	Time from DAG trigger to vLLM hot-swap done	< 2 hours (LoRA) / < 4hr (full)
New data trigger latency	Time from data labelled to fine-tune triggered	< 24 hours
LoRA hot-swap downtime	Requests failing during adapter load	Zero (hot-swap is non-disruptive)
Pipeline artifact size	PVC storage used per fine-tune run	Track; alert if >50 GB/run
Fine-tune cost per run	GPU-hours x \$/hr (track in Airflow metadata)	Baseline + alert on 2x deviation

Shell Verification Commands

```

# 1. Trigger Airflow DAG manually and watch logs
airflow dags trigger nightly_eval_promote
airflow dags list-runs -d nightly_eval_promote --state running
airflow tasks logs nightly_eval_promote quality_gate <run_id>

# 2. Check KFP pipeline runs
kubectl get pods -n kubeflow -l pipeline/runid=<run-id>
# or via UI: http://localhost:8080/runs

# 3. Verify LoRA adapters loaded in vLLM
curl http://localhost:8000/v1/models | python -m json.tool | grep '"id"'

# 4. Test hot-swap -- compare responses from base vs adapter
BASE_MODEL="meta-llama/Meta-Llama-3-8B-Instruct"
ADAPTER="lora-adapter-v1"
PROMPT='{"messages":[{"role":"user","content":"What is KV caching?"}],"max_tokens":60}'

echo "=== Base model ==="
curl -s http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model": "$BASE_MODEL", $PROMPT}' | python -m json.tool

echo "=== Fine-tuned adapter ==="

```

```
curl -s http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"model": "$ADAPTER", $PROMPT}' | python -m json.tool
```

```
# 5. Check Airflow task XCom values (quality scores)
airflow tasks xcom-get nightly_eval_promote eval_production_model <run_id> score_model-v1
airflow tasks xcom-get nightly_eval_promote eval_candidate_model <run_id> score_model-v2
```

Key Takeaways — Day 14 & Full Curriculum

- **Airflow gates the ops loop:** nightly eval -> quality comparison -> auto-promote or Slack alert. The quality gate threshold is the single number that guards every production model change -- set it conservatively and adjust based on your task's acceptable regression tolerance.
- **KubeFlow Pipelines assign GPU resources per step:** fine-tune gets 8xA100, eval gets 1xA10G, deploy is CPU-only. This is the right model for cost efficiency in training pipelines -- never leave 8 GPUs idle during a CPU-bound eval step.
- **LoRA is the production-efficient fine-tuning primitive:** 100MB adapter vs 16GB full model, hot-swappable in vLLM with zero downtime, and multiple adapters can be loaded simultaneously for multi-tenant serving.
- **The continuous pipeline loop:** label data -> accumulate 500+ samples -> trigger KFP run -> eval gate -> hot-swap if passing -> Airflow verifies in next nightly run. This closes the model improvement loop without any human in the critical path.
- **The full 14-day stack in one sentence:** local GGUF/HF inference -> quantization (AWQ/GGUF) -> FastAPI serving -> Docker/K8s -> vLLM with PagedAttention + Flash Attention + continuous batching -> distributed TP/PP -> semantic/prefix caching + API gateway + multi-model routing -> Locust load test + Prometheus/Loki/Jaeger observability -> Airflow eval pipeline + KubeFlow fine-tuning -> LoRA hot-swap. **You now own the entire stack.**

Curriculum Complete. Draw your end-to-end architecture diagram from memory. Every component you cannot place precisely is a topic to revisit. The diagram is your knowledge audit.